



**ÉCOLE DE
TECHNOLOGIE
SUPÉRIEURE**

Université du Québec

Laboratoire 0 – Rapport de laboratoire

Architecture logicielle

LOG430 – Été 2026

Groupe 02

Vincent de Grandpré

DEGV03078209

Remis le vendredi 15 mai 2025 à Montréal

Table des matières

Table des matières.....	2
Introduction et contexte.....	3
Question 1.....	4
Question 2.....	6
Question 3.....	8

Introduction et contexte

Le projet calculatrice est une application monolithique permettant de faire des opérations simples sur des entiers (addition, soustraction, multiplication et division) à partir d'un seul module. La mise en œuvre et la maintenance rapide sont possibles grâce à une structure centralisée et l'utilisation d'un système de conteneurs.

Le développement, l'intégration et le développement continu sont utilisés pour offrir une infrastructure garantissant la simplicité, la maintenabilité et la facilité de déploiement, notamment en standardisant l'environnement de développement, de test et d'exécution avec la conteneurisation.

Conformément aux exigences du laboratoire exprimé dans la Documentation d'architecture (2025-09-05), l'application calculatrice est :

- Testable - par des tests automatisés pour toutes les fonctions, l'exécution des tests dans la chaîne de production CI/CD avec l'utilisation de *pytest* pour générer des sorties détaillées ;
- Déployable – par l'utilisation de GitHub Workflows pour le déploiement, dans différents environnement, avec des étapes d'agencement, de récupération de version et de lancement par le biais d'exécuteurs *GitHub Runner* en mode de service système;
- Maintenable – par du code simple et bien documenté, structuré, cohérent et découplé des contraintes environnementales d'exécution

D'autres attributs de qualité émergent aussi de cette architecture :

- Portable – par l'utilisation systématique de conteneurs indépendamment de l'environnement physique ou virtuel hôte, déploiement sur Ubuntu 22.04 et sur Debian 13 lorsque la première ne fonctionne pas (ce qui arrive souvent) ;
- Observabilité – par l'utilisation d'outils tel que *top* et les journaux systèmes pour consulter les traces d'exécution et les journaux systèmes

Ce projet fait aussi appel à des concepts transversaux tel que l'utilisation de système Linux et de l'outil de connexion SSH.

La portion déploiement continu a généré quelques défis dans le projet alors que l'infrastructure mise à disposition par l'université ne répondait plus à certains moments. Pour permettre d'avancer sur le projet, une autre machine virtuelle a été utilisée, en plus du déploiement via LXC, pour finaliser les tests de portabilité et compléter la rédaction de ce rapport.

Durant le projet, il a été compris que les conteneurs s'utilisent sur des hôtes n'ayant pas le même système d'exploitation et fournissent le même environnement aux applications. Cette portabilité offre un avantage puissant alors qu'une même application peut être déployée sur des systèmes physiques hétérogènes.

En abstrayant l'environnement de l'application de manière homogène dans un conteneur, et en utilisant du code modulaire, il est simple de développer et maintenir l'application calculatrice et de la déployer facilement sur des machines virtuelles.

C'est ce qui a été fait dans ce laboratoire. Les sections suivantes offrent des réponses aux questions posées.

Question 1

« Si l'un des tests échoue à cause d'un bug, comment pytest signale-t-il l'erreur et aide-t-il à la localiser ? Rédigez un test qui provoque volontairement une erreur, puis montrez la sortie du terminal obtenue. »

Réponse : Lorsqu'un test échoue à cause d'un bogue, *pytest* signale l'erreur de quatre façons.

Une première (1), en format fil d'Ariane, où un « . » illustre un test réussi et un F rouge illustre un test en échec. Cet affichage montre la séquence des résultats de tests dans une chaîne de caractère suivant l'ordre des tests.

Une seconde (2), détaillée, identifie l'emplacement de l'erreur dans le code source, souligné par des caractères « ^ » pour la mettre en évidence. Sous cette identification, *pytest* affiche en rouge le détail sur une ligne qu'il préfixe par le caractère **E**, suivi du type de l'erreur et de sa description. À la fin de ce bloc détaillé, le chemin absolu du fichier source, suivi du caractère « : » et du numéro de ligne de l'erreur est affiché.

Une troisième (3) affiche un résumé de l'erreur rencontrée, sur une seule ligne de test, où les informations des erreurs sont présentées dans un format concis, mais en omettant le numéro de ligne et l'emplacement de l'erreur dans la fonction.

Enfin (4), le nombre d'échecs est affiché tout en bas du rapport de test.

Dans les étapes du laboratoire, l'ajout du test *test_addition* tel que fourni contient déjà une erreur que l'on conserve volontairement afin de présenter le résultat du test en échec selon ce qui vient d'être expliqué. En effet, la méthode *addition(int, int)* est méthode de l'objet *Calculator*.

```
def test_addition():  
    assert addition(2, 3) == 5
```

Erreur délibérée dans un test du fichier test_calculator.py

```
# pytest  
===== test session starts =====  
platform linux -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0  
rootdir: /app  
configfile: pyproject.toml  
collected 2 items  
  
src/tests/test_calculator.py .F | 1 [100%]  
  
===== FAILURES =====  
test_addition  
  
def test_addition():  
>     assert addition(2, 3) == 5      2  
           ~~~~~  
E       NameError: name 'addition' is not defined  
  
src/tests/test_calculator.py:15: NameError  
  
===== short test summary info =====  
FAILED src/tests/test_calculator.py::test_addition - NameError: name 'addition' is not defined | 3  
===== 1 failed, 1 passed in 0.29s =====
```

*Exemple d'erreur au premier lancement de pytest dans le projet avec l'ajout d'un appel à la méthode inexistante *addition(int, int)**

Ensuite, un autre test a été volontairement modifié pour générer une erreur et les mêmes signalement d'affichage s'appliquent.

```
# pytest
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0
rootdir: /app
configfile: pyproject.toml
collected 5 items

src/tests/test_calculator.py .F... [100%]

===== FAILURES =====
----- test_addition -----

def test_addition():
    assert Calculator().addition(2, 3) == 5
    > assert Calculator().addition(255, 3234) == 3490
E     assert 3489 == 3490
E     + where 3489 = addition(255, 3234)
E     +   where addition = <calculator.Calculator object at 0x7f95f63ab010>.addition
E     +     where <calculator.Calculator object at 0x7f95f63ab010> = Calculator()

src/tests/test_calculator.py:17: AssertionError
===== short test summary info =====
FAILED src/tests/test_calculator.py::test_addition - assert 3489 == 3490
===== 1 failed, 4 passed in 0.16s =====
```

Exemple d'erreur avec une comparaison délibérément fausse

Question 2

« Que fait GitHub pendant les étapes de « setup » et « checkout » ? Veuillez inclure la sortie du terminal GitHub CI dans votre réponse. »

Durant l'étape préliminaire « setup », GitHub agence un environnement d'exécution selon les spécification du fichier `.github/workflow/ci.yml`. Dans le cas de ce laboratoire, un système d'exploitation virtuel Ubuntu 24.04 est invoqué dans Microsoft Azure à partir d'images fournies par GitHub, les permissions sont accordées au GITHUB_TOKEN, un dossier est créé dans le système de fichiers pour contenir les artefacts du « workflow », puis les « GitHub Actions » référencés dans le fichier `.yml` (`checkout@v3`, `setup-python@v4`) sont installées dans ce système.

```
build
succeeded now in 11s

▼ Set up job

1 Current runner version: '2.334.0'
2 ▼ Runner Image Provisioner
3   Hosted Compute Agent
4   Version: 20260213.493
5   Commit: 5c115507f6dd24b8de37d8bbe0bb4509d0cc0fa3
6   Build Date: 2026-02-13T00:28:41Z
7   Worker ID: {6e5a4ad9-8954-4d4b-a73c-c6f14b43b0e7}
8   Azure Region: northcentralus
9 ▼ Operating System
10  Ubuntu
11  24.04.4
12  LTS
13 ▼ Runner Image
14  Image: ubuntu-24.04
15  Version: 20260413.86.1
16  Included Software: https://github.com/actions/runner-images/blob/ubuntu24/20260413.86/images/ubuntu/Ubuntu2404-Readme.md
17  Image Release: https://github.com/actions/runner-images/releases/tag/ubuntu24%2F20260413.86
18 ▼ GITHUB_TOKEN Permissions
19   Contents: read
20   Metadata: read
21   Packages: read
22 Secret source: Actions
23 Prepare workflow directory
24 Prepare all required actions
25 Getting action download info
26 Download action repository 'actions/checkout@v3' (SHA:f43a0e5ff2bd294095638e18286ca9a3d1956744)
27 Download action repository 'actions/setup-python@v4' (SHA:7f4fc3e22c37d6ff65e88745f38bd3157c663f7c)
28 Complete job name: build
```

Étape « setup » de l'intégration continue de GitHub lors des modifications entrantes (push)

Durant l'étape utilisateur « Checkout dépôt », l'action GitHub `checkout@v3` est utilisée pour télécharger le code source du projet dans l'environnement virtuel créé à l'étape précédente. Pour ce faire, l'outil `git` est configuré pour utiliser un dossier de travail de façon sécuritaire, puis ce dossier est vidé et initialisé par `git`. Ensuite, le projet est cloné après des étapes de configuration de sécurité et le code source est configuré pour suivre la dernière version de la branche `main`.

```
build
succeeded 1 minute ago in 11s

> ✓ Set up job

✓ Checkout dépôt

1 ▶ Run actions/checkout@v3
14 Syncing repository: vdegrandpre/log430-labo0-E26
15 ▶ Getting Git version info
19 Temporarily overriding HOME='/home/runner/work/_temp/aa2e3e56-6a0f-40b6-827e-cdccc0814066' before making global git config changes
20 Adding repository directory to the temporary git global config as a safe directory
21 /usr/bin/git config --global --add safe.directory /home/runner/work/log430-labo0-E26/log430-labo0-E26
22 Deleting the contents of '/home/runner/work/log430-labo0-E26/log430-labo0-E26'
23 ▶ Initializing the repository
40 ▶ Disabling automatic garbage collection
42 ▶ Setting up auth
48 ▶ Fetching the repository
119 ▼ Determining the checkout info
120 ▼ Checking out the ref
121 /usr/bin/git checkout --progress --force -B main refs/remotes/origin/main
122 Switched to a new branch 'main'
123 branch 'main' set up to track 'origin/main'.
124 /usr/bin/git log -1 --format='%H'
125 'df5d36fbcdb9d30efbdee7208d52825268224340'
```

Étape « Checkout dépôt » de l'intégration continue de GitHub lors des modifications entrantes

Le fichier GitHub Workflow a ensuite été amélioré pour permettre le déploiement vers un serveur pouvant être une machine virtuelle Ubuntu 22.04, fournie pour le laboratoire, ou une machine virtuelle Debian 13, auto-hébergée. Pour réussir, la directive d'utilisation d'un conteneur spécifique doit être utilisée, conforme à l'environnement d'exécution souhaité :

```
# Identique à la construction mais sur un noeud auto-hébergé
deploy:
  runs-on: self-hosted
  # Utilisation d'un conteneur docker pour GitHub Runner
  container:
    image: ubuntu:22.04
  environment: Labo0
```

Extrait du fichier .github/workflows/ci.yml pour le déploiement

← LOG430-Labo0_CI

✓ Documentation commentaires #24 Re-run all jobs ...

Summary

All jobs

build

✓ deploy

Run details

Usage

Workflow file

Annotations

1 warning

deploy

succeeded 13 minutes ago in 15s

Search logs

Set up job 3s

Initialize containers 4s

```
1 ▶ Checking docker version
8 ▶ Clean up resources from previous jobs
11 ▶ Create local container network
14 ▶ Starting job container
31 ▼ Waiting for all services to be ready
```

La tâche de déploiement inclut une étape de conteneurisation

La **portabilité** émerge de ce déploiement, qui s'exécute tant sur Ubuntu 22.04 que Debian 13.

LOG430 – E26 – G2 – Labo0 – Rapport – DEGV03078209

Question 3

« Quel type d'informations pouvez-vous obtenir via la commande `top` ? Veuillez donner quelques exemples. Veuillez inclure la sortie du terminal dans votre réponse. »

L'outil en ligne de commande `top` affiche les propriétés de processus Linux, un peu comme le fait le Gestionnaire de tâches de Windows. On retrouvera les données sommaires de l'utilisation des processeurs et de la mémoire (vive et d'échange - *swap*). `Top` affiche ainsi la somme des processus, comprenant les actifs, les dormants (en veille), les arrêtés et les zombies (exécution terminée mais toujours présent dans la table des processus). La ligne de chaque processus la charge du processeur (%CPU) et de la mémoire (%MEM), le numéro du processus et de son parent (PID et PPID), le temps d'exécution total sur les processeurs (TIME+), sa priorité (PRI, numérique ou *rt* pour temps réel), la valeur du *nice* (NI, sert à ajuster la priorité d'exécution dans le processeur) qui influence la colonne précédente, l'état du processus (S), la mémoire virtuelle consommée (VIRT, tout l'espace adressable), la mémoire résidente (RES, réellement utilisée), l'utilisateur exécutant le processus (UID) et finalement la commande d'exécution (COMMAND).

`Top` offre un affichage dynamique ordonné et il est possible de modifier les paramètres d'affichage et de tri. La touche « ? » permet d'afficher la référence des options disponibles pour cet outil. Par exemple, on peut modifier la couleur d'affichage (« Z ») et afficher la commande complète utilisée pour lancer les processus (« c »).

Enfin, `top` permet de terminer l'exécution d'un processus de façon et de modifier sa valeur *nice* pour influencer la priorité d'exécution.

```
Tasks: 86 total, 1 running, 85 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.3 sy, 0.0 ni, 99.7 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3587.0 total, 2154.7 free, 218.8 used, 1213.5 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used, 3301.6 avail Mem
```

PID	PPID	TIME+	%CPU	%MEM	PR	NI	S	VIRT	RES	UID	COMMAND
3587	3507	0:00.03	0.0	0.2	20	0	S	17832	9112	0	python3 src/calculator.py
3575	3564	0:00.43	0.3	0.1	20	0	R	10880	3972	0	top
3564	3518	0:00.02	0.0	0.1	20	0	S	9152	5276	0	-bash
3518	423	0:00.20	0.0	0.3	20	0	S	17204	10876	0	sshd: root@pts/1
3507	3451	0:00.04	0.0	0.2	20	0	S	9276	5648	0	-bash
3451	423	0:00.04	0.0	0.3	20	0	S	17204	10964	0	sshd: root@pts/0
3450	2	0:00.03	0.0	0.0	20	0	I	0	0	0	[kworker/u64:2-events_unbound]
3225	3201	0:00.06	0.0	0.3	20	0	S	14900	11112	0	python3
3201	1	0:00.54	0.0	0.3	20	0	S	1235348	9744	0	/snap/docker/3505/bin/containerd-shim-runc-v+
2781	555	0:00.00	0.0	0.1	20	0	S	8300	3912	0	/usr/bin/dbus-daemon --session --address=sys+
1528	1407	0:05.59	0.3	1.4	20	0	S	1868012	51976	0	containerd --config /run/snap.docker/contain+
1476	2	0:00.26	0.0	0.0	20	0	I	0	0	0	[kworker/0:0-events]
1459	2	0:00.12	0.0	0.0	20	0	I	0	0	0	[kworker/u64:1-events_unbound]
1407	1	0:05.62	0.0	2.6	20	0	S	1923836	97324	0	dockerd --group docker --exec-root=/run/snap+
743	1	0:00.01	0.0	0.2	20	0	S	234512	7304	0	/usr/libexec/polkitd --no-debug
739	1	0:00.05	0.0	0.6	20	0	S	296024	20292	0	/usr/libexec/packagekitd
556	555	0:00.00	0.0	0.1	20	0	S	103652	3656	0	(sd-pam)
555	1	0:00.10	0.0	0.3	20	0	S	17104	9888	0	/lib/systemd/systemd --user
427	1	0:00.08	0.0	0.6	20	0	S	110156	21548	0	/usr/bin/python3 /usr/share/unattended-upgra+
423	1	0:00.01	0.0	0.3	20	0	S	15440	9188	0	sshd: /usr/sbin/sshd -D [listener] 0 of 10-1+
397	1	0:00.00	0.0	0.0	20	0	S	6220	1084	0	/sbin/agetty -o -p -- \u --keep-baud 115200,+
388	1	0:00.13	0.0	0.2	20	0	S	15372	7816	0	/lib/systemd/systemd-logind
386	1	0:05.85	0.0	1.0	20	0	S	1326560	37920	0	/usr/lib/snapd/snapd
382	1	0:00.03	0.0	0.2	20	0	S	222404	5640	104	/usr/sbin/rsyslogd -n -iNONE
381	1	0:00.11	0.0	0.5	20	0	S	33076	19284	0	/usr/bin/python3 /usr/bin/networkd-dispatche+
375	1	0:00.18	0.0	0.1	20	0	S	8660	4776	102	dbus-daemon --system --address=systemd: --n+

Exemple d'utilisation de la commande `top`

Par exemple, on voit dans l'image ci-dessus que le processus de la calculatrice utilise à ce moment le processeur pour un total de 3 centièmes de secondes, n'utilise pas le processeur au moment de la capture et consomme 0,2 % de la mémoire vive, soit 9112 kilo-octets pour un maximum alloué de 17832 kilo-octets.

Des variantes plus dynamiques de l'outil `top` existent, soit `htop` et `btop` et valent la peine d'être mentionnées. On pourra afficher plus d'informations avec des visuels plus intéressants.